Project Report

Submitted By: Vinayak Vikram Singh
Registration Number: 25BAI10262
Project Title: SMS Spam Detection Model

Introduction

Spam messages have become an **everyday annoyance** for mobile users. From **fake lottery wins to misleading promotional texts**, these messages not only waste time but can also expose people to scams and phishing attempts. With the growing number of smartphone users, filtering out unwanted SMS content has become more important than ever.
This project focuses on building a **simple, reliable SMS spam detection system** using **machine learning**. The goal is to help users **automatically identify whether a message is genuine or suspicious**, without requiring any technical knowledge.

Problem Statement

People receive a large number of SMS messages daily, and manually identifying which ones are spam is inefficient and often inaccurate. Many spam messages are cleverly written to look legitimate, making it difficult for users to recognize them at first glance. This creates a need for an automated solution that can analyze a text message and classify it as **spam** or **ham** quickly and accurately.
The project aims to solve this by developing an ML-powered classifier that processes SMS text, learns from patterns in real-world spam messages, and delivers instant, dependable predictions.

Functional Requirements

- The system must allow the user to input an SMS message through the console.
- The system must load an existing trained model if available.
- If no model is found, the system must train a new model automatically using the dataset.
- The system must preprocess and vectorize the input SMS using TF-IDF.
- The system must classify the message as either **spam** or **ham**.
- The system must return the predicted label to the user.
- The system must display a confidence score (if available from the model).
- The system must clear the console before each run for a clean interface.
- The system must save the trained model for future use.
- The system must handle empty or invalid input gracefully (e.g., warn or exit).
- The system must run offline once the model and dataset are present.

- The system must process predictions instantly after loading the model.

Non-Functional Requirements

- **Performance:**
  The system should classify an SMS message in under one second after the model is loaded.

- **Usability:**
  The console interface must be simple, clean, and easy for any user to interact with.

- **Reliability:**
  The system must consistently produce accurate predictions and handle unexpected inputs safely.

- **Maintainability:**
  The codebase should be easy to update, with modular components for loading data, training, and prediction.

- **Portability:**
  The project must run on any platform that supports Python (Windows, macOS, Linux).

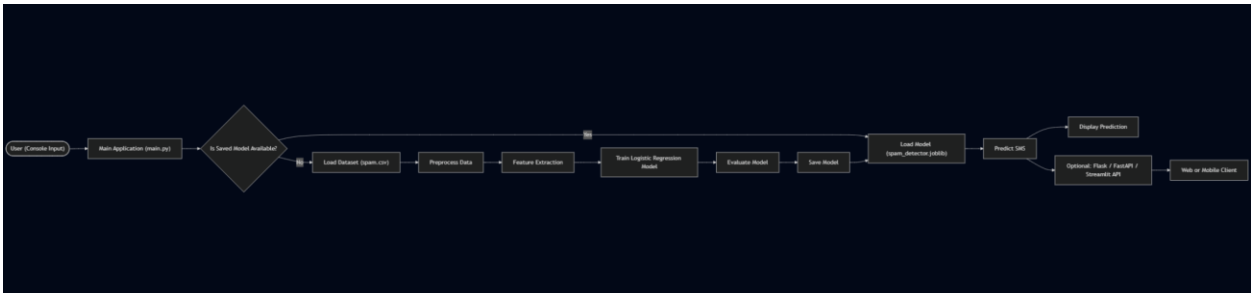
- **Scalability:**
  The system should be able to handle larger datasets or upgraded models in the future without major changes.

- **Security:**
  The system must run offline, ensuring user messages are not shared over the internet.

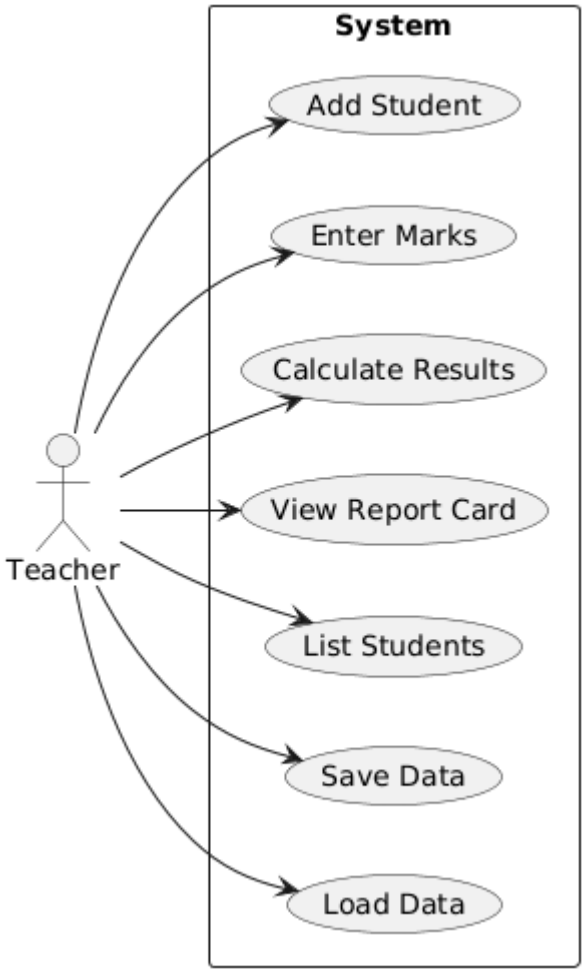
- **Robustness:**
  The application should not crash due to missing files, corrupted data, or empty user input.

- **Storage Efficiency:**
  The trained model file should remain lightweight, suitable for storage in local environments.
-

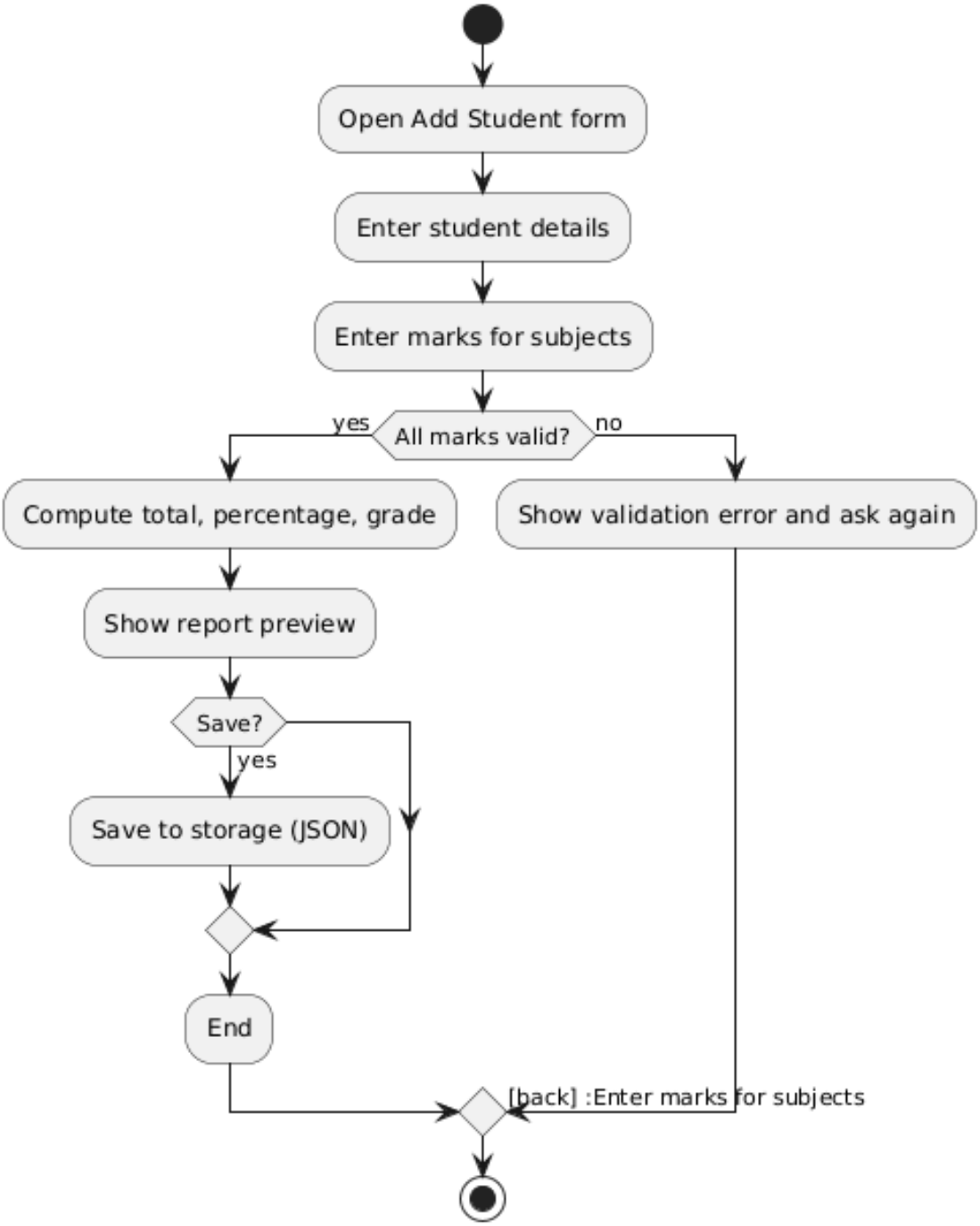
System Architecture



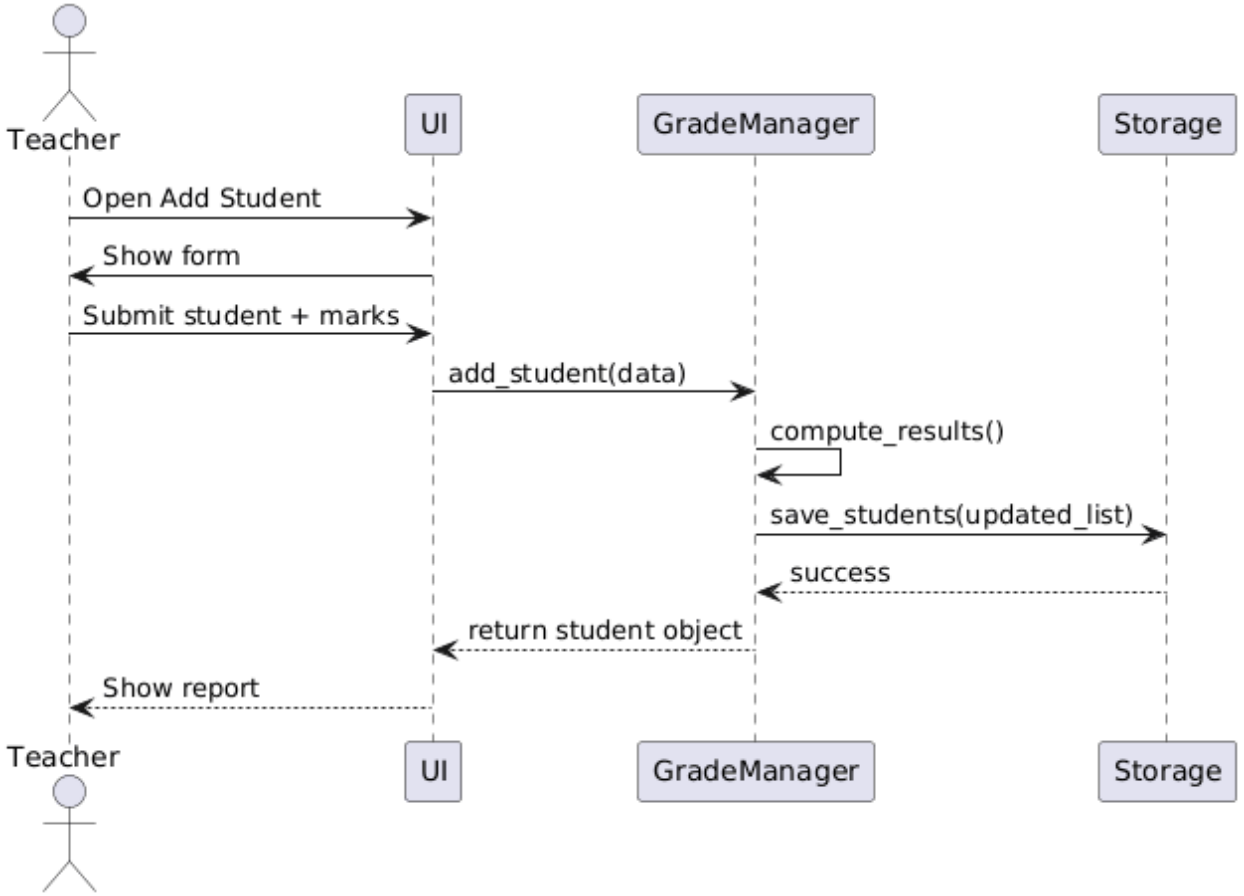
Use Case Diagram



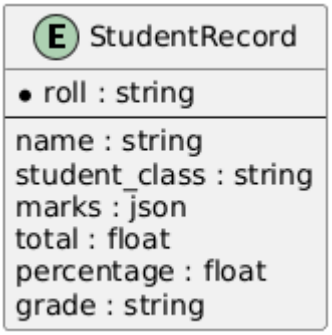
Workflow Diagram



Sequence Diagram



ER Diagram



Design Decisions & Rationale

- **Use of Logistic Regression instead of Naive Bayes**
  Logistic Regression provides better spam detection accuracy and significantly higher recall for spam messages. It handles feature weights more effectively and produces probabilities, making predictions more interpretable.

- **Combining Word-Level and Character-Level TF-IDF Features**
  Word n-grams capture meaningful phrases, while character n-grams detect obfuscated spam patterns like “fr33” or “w1n”. Using both together improves robustness and accuracy, especially against tricky spam messages.

- **Saving the Model Using Joblib**
  Saving the trained model allows instant predictions in future runs without retraining. This reduces computation time, improves usability, and makes the application feel fast and responsive.

- **Building a Console-Based User Interface**
  A CLI was chosen for simplicity, portability, and ease of testing. It allows quick user input, provides clear output, and avoids the overhead of setting up a graphical interface.

- **Automatic Model Training When No Saved Model Is Found**
  This ensures the system is self-contained and user-friendly. New users don’t need technical knowledge—if the trained model doesn’t exist, the application builds one automatically using the dataset.

Implementation Details

The system preprocesses SMS data, extracts features using TF-IDF, and trains a Logistic Regression model to classify messages. The trained model is saved for quick reuse, and the program loads it on each run, accepts a user-entered SMS, processes it, and outputs a spam or ham prediction with confidence.

Screenshots

```
SMS Spam Detector — classify an SMS as 'spam' or 'ham'.

Enter SMS message to classify (or press Enter to quit): HELLO maam

Prediction: Not Spam  (confidence: 92.62%)
(sklearn-env) PS C:\Users\Arjun\Desktop\SMS Spam Detector>
```

Prediction Output - Not Spam

```
SMS Spam Detector — classify an SMS as 'spam' or 'ham'.

Enter SMS message to classify (or press Enter to quit): WIN FREE CAR: WwW.CAR.COM :: xxxx $500

Prediction: spam  (confidence: 75.49%)
(sklearn-env) PS C:\Users\Arjun\Desktop\SMS Spam Detector>
```

Prediction Output - Spam

Challenges Faced

The main challenges included handling inconsistent dataset formatting, improving spam recall without overfitting, and choosing a model that balanced accuracy and performance. Integrating both word and character TF-IDF features also required careful tuning to avoid excessive processing time.

Learnings & Key Takeaways

This project provided hands-on experience with text preprocessing, feature extraction, and building end-to-end machine learning pipelines. It reinforced the importance of model evaluation, vectorization choices, and saving models for efficient reuse in real applications.

Future Enhancements

Future improvements could include building a web or mobile interface, adding deep learning models for better accuracy, expanding the dataset for broader coverage, and enabling batch predictions or real-time SMS filtering.

References

1. www.w3schools.com
2. geeksforgeeks.org
3. vityarthi.com